

DailyMattr CMS

MVP Plan & Database Schema - 25 July 2026

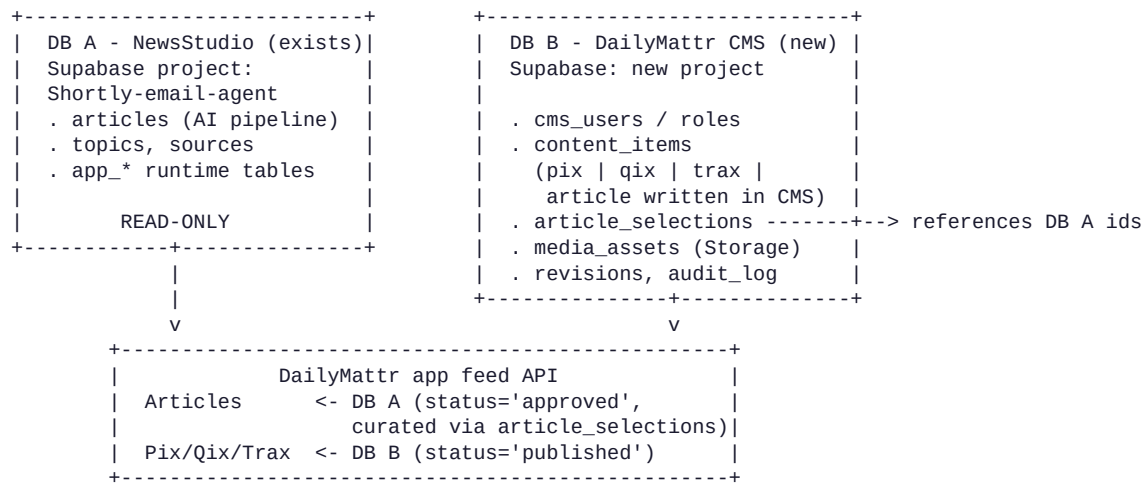
Goal: a role-based CMS (modeled on the Shortly India CMS admin panel) that acts as the uploading platform for the **DailyMattr** app. News **Articles** are pulled from the existing NewsStudio pipeline database (read-only), while **Pix / Qix / Trax** and any originally-written articles are authored and stored in a **new, separate CMS database**.

1. What was verified before writing this plan

Source	What it told us
Shortly CMS admin panel (dridha.webportal.cms, shortlyindia.com)	Role-based CMS with 4 roles: Super Administrator (full access), Chief Editor (language & content management), Content Writer (state-level content), QA (quality review & publish moderation). Content lives under /content/<type> routes (e.g. /content/videos).
dailymattr.com	Content model of the app: Articles (READ - 100 stories/day, 60-word stories, image-first, swipe-through), Qix (WATCH - short video explainers), Trax (LISTEN - audio explainers, podcast-style). Plus Long Mattr (deeper reads, subscription). Pix is a confirmed fourth type: image-format stories.
longmattr.com/general	Article presentation: title, ~90-word summary, category, relative time, fact score (" FACT 100 - 3 SRC "), source attribution, AI views (ELI5, 60-sec TL;DR, Key numbers, Deep dive). Frontend runs on Supabase .
Supabase project Shortly-email-agent (ygxdrphajvrbjcaxhvcn)	This is the NewsStudio articles database. articles table (2,523 rows) with full pipeline: pending -> summarized -> approved -> sent, plus edited_title, edited_summary, category, fact_score, fact_label, fact_notes, image_url, embedding, topics, sources, and app-side tables (app_events, app_seen, app_breaking, app_push_tokens, app_user_vectors).
Supabase project Shortly AI Agent (lplmnnxbpmpijlrorhbt)	Only a published_articles dedup log (source URLs). Not the article store.

Key conclusion: the "specific database the articles should be taken from" already exists - it is the `articles` table in the Shortly-email-agent Supabase project. The CMS treats it as **read-only** and never disturbs the running agent pipeline.

2. Architecture - two databases, one app feed



Why this split (recommended):

- The NewsStudio agent pipeline keeps running untouched; the CMS never writes to DB A.
- Curation of which articles appear in the app is done via a small `article_selections` table **in DB B** that stores DB A article ids - so editorial control over articles also lives in the CMS database.
- Pix / Qix / Trax (and any article written by hand in the CMS) live entirely in DB B.
- The app reads articles from DB A and other content from DB B. If later a single read surface is preferred, add a sync-on-publish step that copies approved DB A articles into DB B `content_items` (`source='newsstudio', source_article_id`) - the schema already supports it: a config change, not a migration.

3. Content types & fields

Type	Mode	Core fields
Article (from NewsStudio)	READ	title, 60-word summary, image, category, fact score/label, source links - already in DB A
Article (written in CMS)	READ	same shape, authored manually in DB B
Pix	READ / visual	image-format story: image card(s) + caption/short text (confirmed by product owner)
Qix	WATCH	short video explainer: video file/URL, thumbnail, duration, topic
Trax	LISTEN	audio explainer: audio file/URL, cover art, duration, chapters/description

All types share: title, slug, summary, category, language, tags, cover image, scheduling, workflow status, author/reviewer, and audit trail.

4. Roles & workflow (mirrors the Shortly CMS)

Roles: **super_admin** - **chief_editor** - **writer** - **qa**

```

draft --> in_review --> approved --> published --> archived
    ^           |
    +- rejected +- (with QA note; writer revises and resubmits)

```

- **Writer:** create/edit own drafts, submit for review, upload media.
- **QA:** review queue, approve or reject with notes.
- **Chief Editor:** everything QA can + edit any item, manage categories, publish/schedule, curate article selections.
- **Super Admin:** everything + user management + settings.

This intentionally matches the existing NewsStudio statuses (pending -> summarized -> approved -> sent) so editors learn one mental model.

5. DB B schema (new Supabase project - Postgres DDL)

```
-- enums -----
create type cms_role      as enum ('super_admin','chief_editor','writer','qa');
create type content_kind  as enum ('article','pix','qix','trax');
create type content_status as enum ('draft','in_review','rejected','approved',
                                     'published','archived');
create type media_kind    as enum ('image','video','audio');

-- users & roles (backed by Supabase Auth) -----
create table cms_users (
  id          uuid primary key references auth.users(id) on delete cascade,
  email       text not null unique,
  full_name   text not null,
  role        cms_role not null default 'writer',
  languages   text[] not null default '{en}', -- chief-editor scoping
  states      text[] not null default '{}',   -- writer scoping (state-level)
  is_active   boolean not null default true,
  created_at  timestampz not null default now()
);

-- taxonomy -----
create table categories (
  slug        text primary key,
  name        text not null,
  kind        content_kind, -- null = applies to all kinds
  sort_order  int not null default 0,
  is_active   boolean not null default true
);

-- media (files live in Supabase Storage buckets) -----
create table media_assets (
  id          uuid primary key default gen_random_uuid(),
  kind        media_kind not null,
  bucket      text not null, -- 'images'|'videos'|'audio'
  storage_path text not null,
  mime_type   text not null,
  size_bytes  bigint,
  width       int,
  height      int,
  duration_sec numeric,
  uploaded_by uuid not null references cms_users(id),
  created_at  timestampz not null default now()
);

-- unified content table (pix / qix / trax / cms articles) -----
create table content_items (
  id          uuid primary key default gen_random_uuid(),
  kind        content_kind not null,
  title       text not null,
  slug        text not null unique,
  summary     text, -- 60-word story text
  body        jsonb not null default '{}', -- kind-specific payload
  category_slug text references categories(slug),
  tags        text[] not null default '{}',
  language    text not null default 'en',
  state       text, -- state-level scoping
  cover_media_id uuid references media_assets(id),
  primary_media_id uuid references media_assets(id), -- video (qix)/audio (trax)
  media_url   text, -- external URL alternative
  duration_sec numeric, -- qix / trax
  source_links jsonb not null default '[]', -- [{title,url}] attribution
  fact_score  numeric, -- parity with NewsStudio
  fact_label  text,
  source      text not null default 'cms', -- 'cms' | 'newsstudio'
  source_article_id uuid, -- DB A articles.id when synced
  status      content_status not null default 'draft',
  review_note text,
  created_by  uuid not null references cms_users(id),
  reviewed_by uuid references cms_users(id),
  published_by uuid references cms_users(id),
  scheduled_at timestampz,
  published_at timestampz,
  created_at  timestampz not null default now(),
  updated_at  timestampz not null default now()
);
```

```

);
create index on content_items (kind, status, published_at desc);
create index on content_items (created_by, status);

-- curation of NewsStudio articles (DB A ids, no writes to DB A) -----
create table article_selections (
  article_id      uuid primary key,           -- DB A articles.id
  title_snapshot  text not null,             -- denormalized for CMS list UI
  image_url       text,
  category        text,
  position        int,
  is_featured     boolean not null default false,
  selected_by     uuid not null references cms_users(id),
  selected_at     timestamptz not null default now(),
  removed_at      timestamptz               -- soft-remove from app feed
);

-- versioning & audit -----
create table content_revisions (
  id              uuid primary key default gen_random_uuid(),
  content_id      uuid not null references content_items(id) on delete cascade,
  snapshot        jsonb not null,           -- full row snapshot at save
  edited_by       uuid not null references cms_users(id),
  created_at      timestamptz not null default now()
);

create table audit_log (
  id              bigint generated always as identity primary key,
  actor_id        uuid references cms_users(id),
  action          text not null,           -- 'create','submit','approve','reject','publish'
  entity          text not null,          -- 'content_item','article_selection','user',...
  entity_id       text not null,
  detail          jsonb not null default '{}',
  created_at      timestamptz not null default now()
);

```

Row-Level Security sketch (enable on every table)

- **cms_users**: user reads own row; super_admin full access.
- **content_items**: writers select/insert/update where created_by = auth.uid() and status in ('draft','rejected'); QA/chief editor read all + update status; only chief_editor/super_admin can set status='published'.
- App feed reads go through API routes using the service key or a dedicated anon-safe view (published_content where status='published'), never raw tables.

Storage buckets: images (5 MB cap), videos (200 MB), audio (100 MB) - private buckets with signed URLs (or public-read for published assets).

DB A access from the CMS: server-side only, read-only. Either (a) a Postgres role in DB A with SELECT on articles only, or (b) DB A's Supabase service key used from CMS API routes with queries restricted to status='approved'.

6. CMS app (MVP surface)

Stack: Next.js (App Router) + Supabase JS (Auth, DB, Storage) + Tailwind/shadcn. Deploy on Vercel. One repo: dailymattr-cms.

Pages (mirroring the reference CMS's /content/<type> structure):

- **/login** - role-aware login (Supabase Auth email/password)
- **/dashboard** - counts by status, recent activity
- **/content/articles** - two tabs: **NewsStudio** (browse DB A approved articles, search, select/feature for the app via article_selections) and **Written in CMS** (authored articles)
- **/content/pix** - **/content/qix** - **/content/trax** - list + editor per type (metadata form + media upload + preview + Save draft / Submit for review)
- **/review** - QA queue: preview, approve / reject with note

- **/users** - super admin: invite users, assign roles/languages/states
- **/settings** - categories management

App-facing API (Next.js routes or Supabase Edge Functions)

- GET `/api/feed/articles?category=&cursor=` - DB A approved articles, ordered by `article_selections.position / published_at`
- GET `/api/feed/{pix|qix|trax}?cursor=` - DB B published items
- GET `/api/content/:id` - single item detail

7. Build phases

Phase	Scope	Exit criteria
1. Foundation (wk 1)	New Supabase project, schema + RLS + buckets, auth, role-gated shell UI	4 role logins work; RLS verified with each role
2. Articles (wk 2)	DB A read-only browser, search/filter, <code>article_selections</code> curation, feed API for articles	Curated article list served by GET <code>/api/feed/articles</code>
3. Authoring (wk 3)	Pix/Qix/Trax editors + media upload; CMS-written articles; drafts & revisions	Writer can create all 4 kinds and submit for review
4. Workflow & publish (wk 4)	QA queue, approve/reject, publish + schedule, feed APIs for pix/qix/trax, audit log	Full draft-to-published path; app can consume all feeds

Post-MVP backlog: sync-on-publish of DB A articles into DB B, push notifications on publish (`app_push_tokens` already exists in DB A), analytics dashboard, multi-language workflows, AI-assist (summarize to 60 words, fact-score) reusing the NewsStudio pipeline.

8. Open questions

- Should app clients eventually read **only one** DB? (Schema supports sync-on-publish.)
- Which categories list should seed DB B - reuse DB A's `newsletter_categories` (12 rows)?
- Reference CMS internals - once logged in to the Shortly CMS, the upload-form layouts can be mirrored exactly.

9. Security note (separate project)

Supabase flagged that the **SignalReply-MVP** project (`irhkzzwkjnzgmrantib`) has RLS disabled on all 14 public tables - anyone with its anon key can read/write every row. Unrelated to this CMS, but worth fixing (enable RLS + add policies; enabling without policies blocks all client access, so plan the policies first).